UNAB-Ambiental

Informe técnico — Laboratorio 4: AMQP + tercer protocolo

IoT + Cloud + Sistemas Distribuidos · 2026

Integrantes: Andi Lin – U00180153 · Juan Sebastián Buitrago Caballero – U00179631 · Miguel Ángel Solano Díaz – U00177983

Objetivos

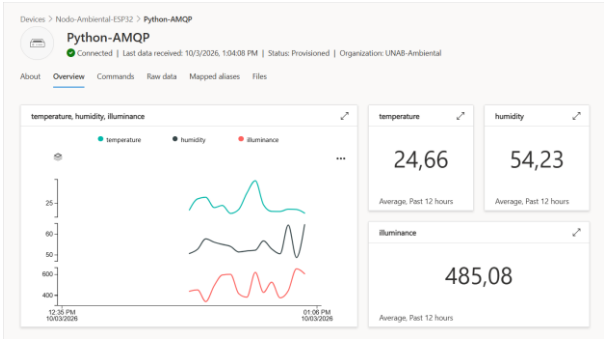
Entender el rol de AMQP en Azure frente a MQTT. Publicar las mismas 3 variables del Lab 3 hacia IoT Central usando transporte AMQP desde la VM. Implementar un tercer protocolo justificado (HTTP/HTTPS). Construir una tabla comparativa MQTT · AMQP · HTTP y recomendar qué usar en cada capa de la plataforma propia.

Rol de AMQP en Azure

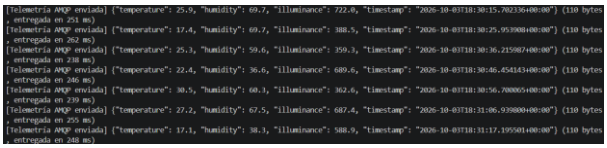
AMQP 1.0 es el protocolo que Azure usa de forma intensiva en el plano de servicio (Service Bus, Event Hubs), con sesiones, enlaces (links) y crédito de flujo que ofrecen garantías de entrega más ricas que el pub/sub ligero de MQTT. El SDK azure-iot-device de Python no soporta AMQP como transporte de dispositivo (solo MQTT); la documentación oficial de Microsoft recomienda la librería uamqp, pero está archivada y falla al compilar en Python moderno. Se usó en su lugar python-qpid-proton (Apache Qpid Proton), la alternativa activamente mantenida que el propio enunciado habilita ("uamqp / proton, o equivalente vigente"), instalada como paquete de sistema (python3-qpid-proton) para obtener soporte TLS correctamente enlazado contra OpenSSL.

Etapa 1 — AMQP hacia IoT Central

Se conectó por AMQP sobre TLS nativo (puerto 5671) a un dispositivo nuevo (Python-AMQP). Un hallazgo clave durante la implementación: por defecto, python-qpid-proton codifica el cuerpo del mensaje como una sección AMQP-VALUE, que IoT Hub no interpreta como payload de telemetría JSON (el dispositivo aparecía Connected pero sin datos). La solución fue activar message.inferred = True con el body en bytes, forzando una sección AMQP-DATA (binaria), equivalente al payload crudo que MQTT transmite por diseño. Esto ilustra justamente el problema que MQTT resuelve de forma más simple: al ser un protocolo con un único tipo de payload (bytes opacos), no exige decisiones de tipificación que sí exige el más rico y flexible AMQP.



Python-AMQP: Connected, 3 variables llegando tras el fix de inferred=True

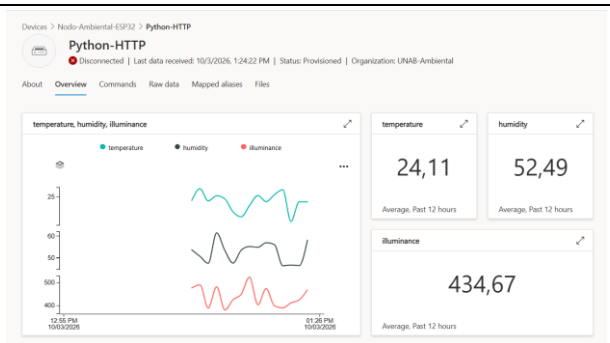


Terminal: mensajes AMQP entregados en ~238-262 ms

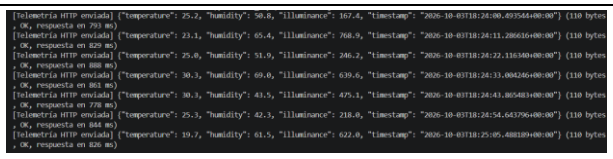
Etapa 2 — Tercer protocolo: HTTP/HTTPS

Se eligió HTTP/HTTPS como tercer protocolo, justificado por una razón de negocio concreta: es el único de los tres que usa el puerto 443 estándar de forma indistinguible de tráfico web normal, por lo que sobrevive sin cambios detrás de firewalls corporativos restrictivos que bloquean 8883 y 5671; además es trivial de depurar con curl o Postman sin ninguna librería especializada. Se implementó un cliente que reutiliza la misma generación de SAS token de los labs anteriores, pero hace un POST HTTPS directo al endpoint nativo /devices/{deviceId}/messages/events de IoT Hub, sin mantener conexión persistente.

Hallazgo relevante: el dispositivo Python-HTTP aparece como Disconnected en IoT Central incluso mientras los datos llegan correctamente y las gráficas se pueblan en tiempo real. Esto no es un error, sino la consecuencia directa de que HTTP no mantiene una conexión persistente: IoT Central no tiene forma de inferir que el dispositivo sigue "en línea" entre una petición y la siguiente, a diferencia de MQTT/AMQP donde el socket persistente permite un estado de conexión confiable.



Python-HTTP: Disconnected pero con datos llegando y graficándose con normalidad



Terminal: mensajes HTTP entregados en ~778-888 ms (OK)

Tabla comparativa: MQTT · AMQP · HTTP

Criterio	MQTT (Lab 3)	AMQP	HTTP
Latencia percibida (medida)	~185-200 ms (QoS 1)	~238-262 ms	~778-888 ms
Overhead / payload	110 bytes + conexión persistente ligera	110 bytes + protocolo más pesado (sesiones, enlaces, créditos de flujo)	110 bytes + cabeceras HTTP completas en cada mensaje, sin reutilizar conexión
Fiabilidad / estado de conexión	Connected persistente y confiable	Connected persistente, igual que MQTT	Disconnected aunque los datos lleguen bien — no refleja estado real
Soporte en microcontrolador/Wokwi	Nativo y liviano, ideal para ESP32	Pesado para la RAM limitada de un microcontrolador	Viable, pero costoso en tiempo/batería por mensaje
Facilidad de debug	Buena: logs claros, topics visibles	Más compleja: hubo que resolver un bug de codificación (inferred=True)	La más fácil: probable con curl/Postman sin librerías especiales
Paso de firewalls restrictivos	Puerto 8883, puede estar bloqueado	Puerto 5671, igual de restringido (443 con AMQP-WS como alternativa)	La mejor: puerto 443 estándar, indistinguible de tráfico web
Caso de uso ideal	Telemetría continua de dispositivos IoT en campo	Integración con Service Bus/Event Hubs, mensajería empresarial	Diagnóstico rápido, redes muy restrictivas, sistemas legados

Recomendaciones de capa

En el dispositivo (Wokwi/ESP32 futuro): MQTT. Es el más liviano, el que mejor soportan los microcontroladores con RAM limitada, y el que demostró mayor estabilidad en Wokwi.

Entre servicios Azure o en la plataforma propia (Labs 5-8): AMQP. Las garantías de entrega más ricas (sesiones, créditos de flujo) son justo lo que Service Bus/Event Hubs necesitan para procesar telemetría a escala entre microservicios internos.

Con cuáles 2 protocolos se quedarían para el resto del curso: MQTT y AMQP. HTTP demostró ser útil como protocolo de emergencia (depuración rápida, redes muy restrictivas), pero su latencia casi 4 veces mayor que MQTT y la pérdida del estado de conexión en tiempo real lo descartan como protocolo principal para un sistema de monitoreo ambiental, donde saber si un nodo sigue activo es parte del valor del producto.

Link de Github

https://github.com/AndiLinUnab/unab-ambiental-lab4.git